

VISÃO DO PRODUTO E DO PROJETO

Versão 1.1

Tabela - Integrantes do Grupo:

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
242004457	Alexandre Henrique Almeida Valadares Sousa	Banco de dados	10
242028655	Davi Kenichi Watanabe Sakai	Frontend	10
241025953	Igor Lima Carneiro	Backend	10
242005329	Jhonatan William Araújo de Almeida	Banco de dados	10
242015432	João Gabriel Rolim Veiga	Backend	10
241039322	João Paulo Jacomini Batista	Frontend	10
241039304	João Victor Amorim Kurihara	Frontend	10
242024253	Lucas Ferreira Santana	Backend	10
242024271	Lucas Peixoto Rodrigues	Backend	10
242005006	Rafael de Aquino Marinho	Frontend	10

Histórico de Revisões

Data	Versão	Descrição	Autor
30/04/2026	1.0	Criação da primeira versão do documento	Grupo Dijkstra
02/06/2026	1.1	Adicionado menções diretas às fontes bibliográficas usadas no corpo do documento e atualização dos sprints	Grupo Dijkstra

--	--	--	--

SUMÁRIO

1 VISÃO GERAL DO PRODUTO.....	4
1.1 Problema.....	4
1.2 Declaração de posição do produto.....	5
Tabela 1 – Posicionamento estratégico do produto.....	5
Fonte: realizado pelo autor (2026).....	5
1.3 Objetivos do Produto.....	5
1.4 Tecnologias a Serem Utilizadas.....	5
2 VISÃO GERAL DO PROJETO.....	7
2.1 Ciclo de vida do projeto de desenvolvimento de software.....	7
2.1.1 Metodologia: Metodologia Ágil.....	7
2.1.2 Processo:.....	7
2.1.4 Métodos:.....	7
2.1.5 Ferramentas:.....	8
2.2 Organização do Projeto.....	8
2.3 Planejamento das Fases e/ou Iterações do Projeto.....	9
2.4 Matriz de Comunicação.....	10
2.5 Gerenciamento de Riscos.....	10
2.6 Critérios de Replanejamento.....	11
3 PROCESSO DE DESENVOLVIMENTO DE SOFTWARE.....	12
3.1 Principais Práticas Adotadas.....	12
3.2 Ferramentas de Suporte.....	12
4 DECLARAÇÃO DE ESCOPO DO PROJETO.....	14
4.1 Backlog do produto.....	14
4.2 Perfis.....	15
4.3 Cenários.....	15
5 MÉTRICAS E MEDIÇÕES.....	18
5.1 GQM de medições.....	18
6 TESTES DE SOFTWARE.....	20
6.1 Estratégia de testes contendo:.....	20
6.1.1 Testes implementados em níveis.....	20
6.1.2 Testes funcionais e não funcionais.....	20
6.1.3 Ambientes de Teste e Política de Branches e Commits.....	20
6.1.3.1 Ambientes de Teste.....	21
6.1.3.2 Política de Branches.....	21
6.1.3.3 Política de Commits e Integração.....	21
6.1.4 Análise dos Testes.....	22
6.2 Roteiro de teste:.....	22
7. REFERÊNCIAS BIBLIOGRÁFICAS.....	24

1 VISÃO GERAL DO PRODUTO

1.1 Problema

Segundo a Associação Brasileira de Bares e Restaurantes (ABRASEL, 2018), aproximadamente 50% dos estabelecimentos do setor encerram suas atividades em menos de dois anos de operação. A causa raiz desse declínio raramente está ligada à qualidade da comida, e sim decorrente de falhas na gestão de pedidos, descontrole de estoque e ineficiência na comunicação interna.

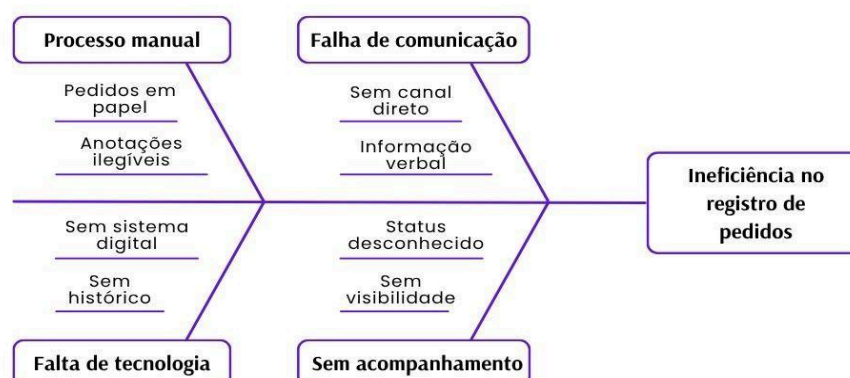
O gerenciamento manual de pedidos, baseado em comandas de papel e interações verbais, está fadado ao erro (ALELO, 2024). O fluxo de informações em um restaurante é dinâmico e demanda alta carga cognitiva, onde a perda de um único pedaço de papel pode significar a interrupção de toda a experiência do cliente e a perda direta de receita (CLOUDFY, 2025).

A anotação em papel é uma interface de entrada de dados de baixíssima fidelidade, sujeita a ambiguidades de caligrafia, falta de padronização em observações e total ausência de dados temporais. Sem o registro exato do momento em que o pedido foi feito, a cozinha perde a capacidade de medir o tempo médio de preparo, informação vital para a eficiência operacional do restaurante uma vez que a agilidade é um dos fatores mais valorizados pelos consumidores modernos.

Focando na falta de rastreabilidade, falhas de comunicação entre salão e cozinha e desorganização logística, conforme ilustrado na Figura 1, revela-se a necessidade de desenvolver um software que solucione a problemática do registro e acompanhamento de pedidos.

O sistema deve permitir a entrada de pedidos via dispositivos móveis (garçons) e enviá-los imediatamente para telas na cozinha, organizando-os por prioridade e tempo de preparo, eliminando a necessidade de impressoras, além de possuir interfaces intuitivas que exijam o mínimo de treinamento possível para a equipe, considerando a alta rotatividade de funcionários no setor de *foodservice*.

Figura 1 - Ineficiência no registro e acompanhamento de pedidos



Fonte: realizado pelo autor (2026)

1.2 Declaração de posição do produto

A tabela 1 condensa o posicionamento estratégico do Nexus Gourmet, apresentando seu público-alvo, necessidade no ambiente, sua categoria e vantagens em relação às alternativas existentes.

Tabela 1 – Posicionamento estratégico do produto

Para:	Proprietários e funcionários de restaurantes de qualquer porte
Necessidade	Registrar, acompanhar e gerenciar pedidos de forma centralizada, ágil e sem erros
O Nexus Gourmet:	É uma aplicação WEB - mobile denominada Nexus Gourmet
Que:	Permite o registro digital de pedidos por mesa, o acompanhamento do status em tempo real pela cozinha e o fechamento de conta pelos garçons
Ao contrário:	Do registro manual em papel e da comunicação verbal entre garçom e cozinha, que estão sujeitos a erros, perdas de informação e ausência de rastreabilidade
Nosso produto:	Integra o fluxo completo do pedido — do salão à cozinha — com atualização em tempo real, dispensando qualquer infraestrutura adicional além de um navegador web

Fonte: realizado pelo autor (2026)

1.3 Objetivos do Produto

- Objetivo Principal

O Nexus Gourmet objetiva, principalmente, desenvolver um software que centralize o processo de registro, acompanhamento e gerenciamento de pedidos de restaurantes, eliminando a dependência de anotações manuais.

- Objetivos Secundários

- Melhorar o gerenciamento de insumos consumidos pelo cliente.
- Oferecer visibilidade em tempo real do status de cada pedido (NOX, 2025; OLITECNICA, 2025).
- Facilitar o gerenciamento de mesas (THEFORKMANAGER, 2025).
- Automatizar o cálculo e a geração da conta ao final do atendimento.

1.4 Tecnologias a Serem Utilizadas

Frontend: HTML, CSS & JavaScript

Backend: Python;

Banco de Dados: MySQL;

Frameworks/Bibliotecas: Flask;

Ferramentas adicionais: GitHub, Microsoft Word, Visual Studio Code.

2 VISÃO GERAL DO PROJETO

2.1 Ciclo de vida do projeto de desenvolvimento de software

Para o desenvolvimento do sistema de gerenciamento de pedidos, adotamos uma abordagem embasada nas práticas ágeis. Essa escolha se justifica pela necessidade de entregas contínuas, validação constante com os usuários finais e a capacidade de adaptação a requisitos dinâmicos característicos do setor de *foodservice*. A seguir, detalhamos a instanciação do ciclo de vida do projeto com base na arquitetura de metodologias:

2.1.1 Metodologia: Metodologia Ágil.

A escolha pela agilidade se dá pelo foco na entrega de valor e pela flexibilidade. Como o ambiente de restaurantes exige alta usabilidade e eficiência operacional, uma abordagem ágil permite que o grupo ajuste prioridades no Backlog do Produto e planeje o escopo conforme os riscos são identificados e mitigados, sem engessar o desenvolvimento.

2.1.2 Processo:

Utilizaremos um processo orientado pelo **Scrumban** (uma abordagem híbrida que combina a estrutura do Scrum com o fluxo contínuo do Kanban), apoiado por práticas de engenharia do **XP** (*Extreme Programming*). O Scrumban nos permite manter os papéis definidos (Dono do Produto, Desenvolvedores e Analistas de Qualidade) e o planejamento em ciclos curtos de uma semana (Sprints), mas com a adoção de um fluxo contínuo (sistema pull) e limites de trabalho em andamento (WIP – *Work in Progress*) para evitar gargalos na equipe.

2.1.3 Procedimentos:

O trabalho fluirá através de iterações baseadas em Sprints, iniciando com uma *Sprint Planning* para puxar as tarefas prioritárias (como as funcionalidades *Must*) para o quadro de desenvolvimento. O acompanhamento diário e o gerenciamento de riscos serão guiados visualmente pelo quadro Kanban, garantindo transparência do que está a fazer, em andamento e concluído. Ao final do ciclo, as entregas passarão por avaliações de qualidade baseadas nas métricas do GQM.

2.1.4 Métodos:

- **Quadro Kanban e Limites de WIP:** Gestão visual do fluxo de tarefas para identificar rapidamente impedimentos e limitar a quantidade de itens em desenvolvimento simultâneo, garantindo foco na conclusão.
- **Histórias de usuário:** Utilizadas para mapear o Backlog do produto de forma focada (Administrador, Garçom, Cozinheiro).

- **Testes de Software:** Implementação de testes para mitigar riscos de comunicação entre interfaces, buscando manter a densidade de erros de programa em níveis mínimos ($\leq 0.5\%$).
- **Refatoração Contínua:** Prática do XP para manter o código limpo e otimizado, assegurando os tempos de execução e de comunicação em tempo real esperados pelo sistema.

2.1.5 Ferramentas:

- **Codificação:** Visual Studio Code utilizando HTML, CSS, JavaScript para o *frontend* e Python com Flask e MySQL para o *Backend*.
- **Design e Prototipagem:** Figma, para desenhar interfaces intuitivas e responsivas.
- **Versionamento e Gestão:** Git, centralizando o código no GitHub, que também poderá ser utilizado para a gestão visual do Kanban.
- **Comunicação da Equipe:** Teams, Discord e WhatsApp.

2.2 Organização do Projeto

A tabela 2 apresenta as atribuições e deveres de cada membro do grupo, ou seja, as responsabilidades escolhidas pelos membros participantes.

Tabela 2 - Divisão de funções

<i>Papel</i>	<i>Atribuições</i>	<i>Responsável</i>	<i>Participantes</i>
Desenvolvedor	Esses membros ficam responsáveis por fazer com que o projeto funcione através das iterações de código e banco de dados	Igor, Davi Sakai, João Gabriel, Jhonatan William	Igor, Davi Sakai, João Gabriel, João Amorim, Jhonatan William
Dono do Produto	Esse membro(s) fica responsável por validar os requisitos e backlog do produto seria como um representante do cliente na equipe	Rafael de Aquino	Rafael de Aquino
Analista de Qualidade	Esses membros ficam responsáveis por avaliar a qualidade do produto e decidir se a iteração está pronta para implementação, de acordo com conceito de pronto do time.	Lucas Peixoto, Alexandre, Lucas Ferreira	Lucas Peixoto, Alexandre, Lucas Ferreira
Cliente (monitor)	Avaliar se o projeto está de acordo com os requisitos e proposta inicial	Lucas Ferreira	Lucas Ferreira

Fonte: realizado pelo autor (2026)

2.3 Planejamento das Fases e/ou Iterações do Projeto

O planejamento de Fases tem como objetivo demonstrar o que foi feito em cada sprint, o período que ela durou e as entregas feitas durante ou ao final da iteração. O planejamento também deixa claro qual o grau de conclusão do projeto. A tabela 3 abaixo apresenta esses dados.

Tabela 3 - Planejamento das fases

Sprint	Produto (Entrega)	Data Início	Data Fim	Entregável(eis)	Responsáveis	% conclusão
Sprint 1	Definição geral do produto	09/04/2026	16/04/2026	-	Todos	2%
Sprint 2	Planejamento do projeto e DV	23/04/2026	30/04/2026	1ª versão do Documento de Visão	Todos	7%
Sprint 3	Documento de arquitetura	30/04/2026	14/05/2026	1ª versão do Documento de arquitetura	Todos	13%
Sprint 4	Preparação do Github	20/05/2026	28/05/2026	Pastas e documentos no git	Todos	18%
Sprint 5	Início do Desenvolvimento	28/05/2026	04/06/2026	Models, Controllers, atualização dos documentos	Igor Lima, João Gabriel, Lucas Ferreira, Lucas Peixoto	20%
Sprint 6	Integração dos models, services e	04/06/2026	11/06/2026	Banco de dados + atualização	Alexandre, Igor Lima, Jhonatan, João	45%

	controllers; desenvolvimento do banco de dados			es do backend	Gabriel, Lucas Ferreira, Lucas Peixoto	
Sprint 7	Integração das camadas e desenvolvimento Frontend	11/05/2026	18/06/2026	Entrega da view de login e do administrador	Davi, João Paulo, João Victor, Rafael	65%
Sprint 8						?

Fonte: Elaborado pelo autor (2026)

2.4 Matriz de Comunicação

A tabela 4 apresenta como serão feitas as comunicações entre o grupo e a monitora. Além disso, mostrará quais são as fontes de informação geradas pelo grupo, sobre o projeto, e onde elas estarão disponíveis para a consulta das iterações.

Tabela 4 - Matriz de comunicação

Descrição	Área/ Envolvidos	Periodicidade e	Produtos Gerados
Acompanhamento das atividades em andamento via Teams, WhatsApp e Acompanhamento dos riscos, compromissos, ações pendentes via github	Equipe do Projeto	Semanal	Ata de reunião Relatório de situação do projeto
	Equipe do Projeto	Quinzenal	
Comunicar a situação do projeto	Equipe do Projeto + monitor	Semanal	Ata de reunião, e Relatório de situação do projeto

Fonte: Elaborado pelo autor (2026)

2.5 Gerenciamento de Riscos

O Gerenciamento de Riscos consiste na identificação, avaliação, priorização, tratamento e monitoramento de possíveis ameaças tanto internas quanto externas. O objetivo de aplicar o gerenciamento de riscos no projeto é promover a continuidade, a segurança no trabalho, a tomada de decisões e a redução de custos.

A tabela 5 logo a seguir mostra os principais riscos que podemos vir a enfrentar durante as etapas do projeto, a tabela classifica os riscos em alto, médio ou baixo, além de propor estratégias de mitigação e plano de contingência.

Tabela 5 - Gerenciamento de riscos

Risco	Grão de exposição	Mitigação	Plano de contingência
Alteração de requisitos do backlog do produto após o início da sprint	Alto	Aumentando o período de refinamento do backlog do produto	Reunir os requisitos que já estão acordados e aumentar o grau de prioridade
Falta de comunicação entre as interfaces do sistema	Alto	Implementação de Testes de Software para garantir a redução de atrasos	Revisão completa de todos as unidades do código em larga escala para solução do problema
Dificuldades com as tecnologias usadas no projeto (SQL, Flask, etc.)	Médio	Preparação rápida para a implementação básica dessas tecnologias	Solicitar ajuda externa para o auxílio das atividades (monitores, professores, tutores etc.)

Fonte: Elaborado pelo autor (2026)

¹

2.6 Critérios de Replanejamento

A seguir, a tabela 6 apresenta os critérios de replanejamento e as ações que serão tomadas para replanejar os seguimentos afetados:

Tabela 6 - Critérios de replanejamento

Risco	Critério de Replanejamento	Ação de Replanejamento
Atraso na entrega de funcionalidades	Atraso, igual ou superior a 1 sprint, na entrega de uma funcionalidade em relação ao cronograma	Alterar as prioridades de entrega e ajustar o backlog
Necessidade de alterar o escopo	Funcionalidades Must não estão sendo realizadas de acordo com o cronograma planejado	Replanejar as funcionalidades que seriam trabalhadas em sprints futuras
Falta de comunicação entre os membros do projeto	Dificuldade de comunicação com um membro por 3 dias	Ajustes na divisão e nas responsabilidades atribuídas aos membros

Fonte: Elaborado pelo autor (2026)

¹ Lembrando: plano de mitigação existe para tentar evitar que um risco ocorra. Plano de contingência é acionado se um risco efetivamente ocorre e deve tratar os efeitos do risco já ocorrido.

3 PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

Conforme apresentado na Seção 2.1, o desenvolvimento do sistema Nexus Gourmet será conduzido com base na metodologia ágil, por meio da abordagem híbrida Scrumban, complementada por práticas de engenharia de software oriundas do Extreme Programming (XP). Essa combinação visa proporcionar flexibilidade, organização e melhoria contínua ao longo de todo o ciclo de vida do projeto.

3.1 Principais Práticas Adotadas

- Para garantir a agilidade e a qualidade do software, a equipe adotou as seguintes práticas:
- **Sprint Planning:** Reunião realizada no início de cada Sprint com o objetivo de selecionar, priorizar e detalhar as funcionalidades a serem desenvolvidas, definindo metas e responsabilidades para a equipe.
- **Sprint Review:** Encontro ao término de cada Sprint para apresentação das funcionalidades implementadas e validação junto ao cliente ou monitor.
- **Sprint Retrospective:** Reunião destinada à reflexão sobre o processo de desenvolvimento, identificando pontos de melhoria e oportunidades de aperfeiçoamento.
- **Gestão Visual com Kanban:** Utilização de quadro Kanban para monitoramento contínuo das tarefas, organizadas em colunas que representam o fluxo de trabalho (a fazer, em andamento, em validação e concluído).
- **Limitação de WIP (Work in Progress):** Estabelecimento de limites para o número de tarefas em andamento simultaneamente, a fim de reduzir sobrecarga e evitar gargalos.
- **Pair Programming (XP):** Prática aplicada em funcionalidades críticas ou de maior complexidade, promovendo colaboração, compartilhamento de conhecimento e redução de defeitos.
- **Code Review (XP):** Revisão sistemática do código desenvolvido, garantindo conformidade com padrões de qualidade, legibilidade e manutenção.
- **Testes Contínuos (XP):** Execução de testes unitários, de integração e testes manuais durante todo o desenvolvimento, assegurando a qualidade incremental do produto.
- **Refinamento Contínuo do Backlog:** Atualização periódica do Backlog do Nexus Gourmet com base no feedback do cliente, nas prioridades de negócio e nas necessidades identificadas pela equipe.

3.2 Ferramentas de Suporte

Para apoiar a execução do processo, serão utilizadas ferramentas de colaboração, comunicação, versionamento e documentação:

- **Versionamento:** Git + GitHub
- **Documentação:** GitHub Pages
- **Prototipagem:** Figma
- **Comunicação:** Teams, Discord e WhatsApp

A tabela 7 a seguir detalha as responsabilidades de cada papel dentro do projeto, adaptada para o formato de gestão profissional do sistema.

Tabela 7 - Papéis

Papéis	Responsabilidades	Integrantes
Dono do produto	Responsável por validar os requisitos e o backlog do produto, atuando como representante do cliente na equipe	Rafael de Aquino
Desenvolvedores	Responsáveis pela implementação técnica do sistema, incluindo código, banco de dados e iterações de desenvolvimento	Davi Sakai, Igor Lima, Jhonatan William, João Gabriel, João Amorim e João Paulo
Analistas de Qualidade	Responsáveis por avaliar a qualidade do produto e validar se as iterações estão prontas para implementação conforme o conceito de "pronto" do time	Alexandre Sousa, Lucas Peixoto e Lucas Ferreira
Cliente	Responsável por avaliar se o projeto está de acordo com os requisitos e a proposta inicial do Nexus Gourmet	Lucas Ferreira

A figura 2 abaixo apresenta o ciclo de vida de desenvolvimento adotado para o projeto **Nexus Gourmet**, representando as fases principais de planejamento, desenvolvimento, revisão, retrospectiva e refinamento do backlog.

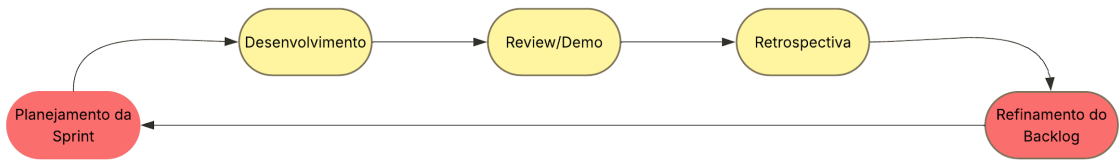


Figura 2 – Ciclo de vida adotado no Nexus Gourmet. Fonte: Elaborada pelos autores (2026)

4 DECLARAÇÃO DE ESCOPO DO PROJETO

4.1 Backlog do produto

A Tabela 8 apresenta o backlog do produto, composto pelo conjunto de funcionalidades identificadas para o sistema. Cada funcionalidade está descrita de forma objetiva e classificada por nível de prioridade, servindo como base para o planejamento das sprints e para o acompanhamento do desenvolvimento ao longo do projeto.

Tabela 8 – Backlog

ID	Funcionalidade	Descrição	Prioridade
F01	Abrir pedido	O garçom deve poder abrir um pedido vinculado a uma mesa	Alta
F02	Adicionar item ao pedido	O garçom deve poder adicionar itens do cardápio a um pedido em aberto	Alta
F03	Remover item do pedido	O garçom deve poder remover itens de um pedido em aberto	Alta
F04	Enviar pedido para a cozinha	O garçom deve poder enviar o pedido, alterando seu status para "em preparo"	Alta
F05	Visualizar pedidos na cozinha	O cozinheiro deve poder visualizar todos os pedidos ativos organizados por status	Alta
F06	Visualizar tempo de espera	O cozinheiro deve poder visualizar o tempo decorrido desde a abertura de cada pedido	Alta
F07	Atualizar status do pedido	O cozinheiro deve poder marcar um pedido como "em preparo" ou "pronto"	Alta
F08	Fechar conta	O garçom deve poder fechar a conta de uma mesa, gerando o total e liberando a mesa	Alta
F09	Calcular total do pedido	O sistema deve calcular automaticamente o valor total do pedido com base nos itens e quantidades	Média
F10	Visualizar mesas	O sistema deve exibir todas as mesas do restaurante com seus respectivos status	Média
F11	Cadastrar produtos	O administrador deve poder cadastrar novos produtos no cardápio, informando nome, categoria e preço	Alta
F12	Editar produto	O administrador deve poder editar as informações de um produto já cadastrado	Média
F13	Remover produto	O administrador deve poder remover um produto do cardápio	Média
F14	Cadastrar mesa	O administrador deve poder cadastrar novas mesas, informando número e capacidade	Alta

F15	Editar mesa	O administrador deve poder editar as informações de uma mesa já cadastrado	Média
F16	Remover mesa	O administrador deve poder remover uma mesa do sistema	Média

Fonte: Elaborado pelo autor (2026)

4.2 Perfis

A Tabela 9 apresenta os perfis de usuário do sistema, descrevendo o papel de cada ator no contexto do restaurante e as responsabilidades a ele atribuídas dentro da aplicação. A definição dos perfis orienta a construção das user stories e dos casos de uso detalhados nas seções seguintes.

Tabela 9: Perfis de acesso

#	Nome do perfil	Características do perfil	Permissões de acesso
P01	Administrador	Responsável pela gestão operacional do sistema	Cadastrar, editar e remover produtos do cardápio e mesas do restaurante
P02	Garçom	Funcionário responsável pelo atendimento no salão	Abrir pedidos, registrar itens, enviar pedidos para a cozinha e fechar contas
P03	Cozinheiro	Funcionário responsável pelo preparo dos pedidos	Visualizar pedidos ativos, acompanhar tempo de espera e atualizar status dos pedidos

Fonte: Elaborado pelo autor (2026)

4.3 Cenários

A Tabela 10 organiza exemplos práticos de como os usuários interagem com o sistema, conectando requisitos a situações reais de uso. Ela orienta o desenvolvimento das funcionalidades e evita desvios do escopo. Os itens não possuem sprint definida, pois foram deixados para planejamentos futuros, reduzindo expectativas irreais. A tabela seguinte apresenta esses cenários, com ator, contexto, passos e resultado esperado:

Tabela 10: Cenários funcionais

Sistema: xxx – Cenários funcionais				
N	Ator	Contexto	Passos	Sprints
1	Administrador	Deseja registrar uma nova mesa	Acessar "Cadastro de Mesas", preencher número,	

			capacidade e salvar.	
2	Garçom	Deseja receber o pedido de uma mesa	Selecionar a mesa desejada, clicar em “abrir pedido” e selecionar os itens desejados	
3	Cozinha	Deseja despachar um pedido finalizado	Selecionar o pedido e clicar em “pronto”, para que o garçom venha retirar	

Fonte: Elaborado pelo autor (2026)

4.4 Tabela de Backlog do Produto

Tabela 11: Backlog do produto

Sistema: xxxx – Backlog do produto						
Numeração (Cenário / requisito)	Sprint	Nome do requisito	Tipo de requisito (Funcional / não funcional)	Priorização do requisito Must, Should, Could	Descrição sucinta do requisito	User histories (U.S.) associadas Identifique as U.S. associadas ao requisito

5 MÉTRICAS E MEDIÇÕES

5.1 GQM de medições

O *Goal Question Metrics (GQM)* foi elaborado com o intuito de estabelecer, através da tabela 12 as métricas do projeto, seguindo seu objetivo principal: desenvolver um software que centralize o processo de registro, acompanhamento e gerenciamento de pedidos de restaurantes.

As métricas foram definidas com base em:

1. Expectativas dos Stakeholders: Entrega de testes intermediários e produto funcional.
2. Riscos do projeto: Comunicação da equipe, cumprimento de prazos e qualidade do código.

Tabela 12 - GQM do produto

Objetivo	Pergunta	Métrica	Cálculo	Escala	Valor esperado	Forma de análise
Validar qualidade de implementação	O software está bem implementado?	Densidade de erros de programa	$(\text{Linhas com erro})/(\text{total de linhas}) \times 100$	%	$\leq 0.5\%$	Implementação de Testes de Software e Refatoração
Eficácia dos alertas visuais	Os alertas estão funcionando corretamente?	Quantidade de falsos positivos	$(\text{Alertas falsos})/(\text{Total de alertas}) \times 100$	%	$\leq 0.2\%$	Por testes de software, registrando a média de alertas falsos
Obediência ao período dos sprints	Os sprints estão entregando todos os musts?	Densidade de prorrogações de musts	$(\text{Quantidade de musts prorrogados})/(\text{Total de Musts do Sprint}) \times 100$	%	100%	Implementação de Sprint Reviews e Sprint Meetings
Usabilidade da Interface de usuário	A interface é intuitiva e perfeitamente utilizável?	Densidade de feedback negativo	$(\text{Reports negativos})/(\text{Total de Reports}) \times 100$	%	$\leq 2\%$	Reuniões de alinhamento de requisitos e contato com o usuário

Verificar quantidade de pedido por garçom	Qual garçom faz mais pedidos?	Densidade de pedidos por garçom	$(\text{Pedidos realizados para cada um})/(\text{Pedidos totais}) \times 100$	%	$\geq (\text{Quantidade de pedidos particulares})/(\text{Pedidos totais})$	A quantidade de contas abertas por garçom
Verificar tempo médio de preparo por produto	Qual o tempo médio que um produto é entregue?	Tempo de entrega	$(\text{Horário que foi entregue}) - (\text{Horário que foi pedido})$	min	$\leq (\text{Média de entrega do produto})$	O tempo gasto entre a abertura da conta e a entrega desde na mesa
Verificar saída de produto	Qual a porcentagem de saída de cada produto?	Densidade de pedidos do produto	$(\text{Pedido do produto})/(\text{Pedidos totais}) \times 100$	%	$(\text{Pedidos totais})/(\text{Quantidade de produtos})$	Quantidade de vezes que o pedido foi feito
Verificar a quantidade de comandas	Quantas comandas foram abertas?	Quantidade de comandas abertas	-	-	-	-

Fonte: Elaborado pelo autor (2026)

6 TESTES DE SOFTWARE

6.1 Estratégia de testes contendo:

6.1.1 Testes implementados em níveis

Os testes implementados no projeto permitem uma maior confiabilidade do código, e podem ser entendidos em níveis:

- **Testes Unitários:** Testam o código de forma isolada, validando a menor unidade funcional do mesmo, garantindo que cada componente da implementação responda corretamente às entradas fornecidas e eliminam custos de manutenção ao prevenir que bugs pequenos cheguem à produção.
- **Testes de Integração:** Verificam como diferentes partes do programa trabalham em conjunto para compor uma funcionalidade inteira, validam a comunicação e a interface entre dois ou mais módulos do sistema, investigam a existência de bugs que surgem apenas quando componentes isoladas são conectados entre si e testam a interação com elementos externos, como bancos de dados.
- **Testes de Sistema:** Testam e validam o software como um todo, garantindo que o produto final atende a todos os requisitos funcionais e técnicos do projeto. Eles garantem que o produto pronto seja meticulosamente o que foi planejado no escopo e avaliam não só as funções de um sistema, mas o desempenho, estabilidade e segurança.

6.1.2 Testes Funcionais

Testes de software funcionais são fundamentais para validação e análise usabilidade de uma aplicação.

- **Testes Funcionais:** Trata-se de testes que verificam as funcionalidades de uma aplicação, é onde são testadas as regras de negócio: login funcional, bloqueio de sistema, registro de pedidos, execução de tarefas

6.1.3 Ambientes de Teste e Política de Branches e Commits

O projeto adota uma estrutura organizada de ambientes de teste integrada à sua política de versionamento de código, utilizando Git e GitHub como ferramentas principais.

6.1.3.1 Ambientes de Teste

São definidos três ambientes principais ao longo do ciclo de desenvolvimento:

- **Ambiente de Desenvolvimento (Dev):** Utilizado para implementação e testes iniciais das funcionalidades. Os desenvolvedores trabalham localmente com as tecnologias do projeto, incluindo HTML, CSS e JavaScript no frontend, Python com Flask no backend e MySQL como banco de dados.
- **Ambiente de Homologação (QA):** Responsável pela validação das funcionalidades integradas. Neste ambiente, são realizados testes funcionais e não funcionais para garantir que o sistema atenda aos requisitos especificados.
- **Ambiente de Produção (Prod):** Ambiente final onde a aplicação é disponibilizada aos usuários. A documentação do sistema é publicada por meio do GitHub Pages.

6.1.3.2 Política de Branches

A organização das branches segue uma estrutura baseada em fluxo contínuo de integração:

- **main:** Representa o ambiente de produção. Apenas código validado e estável é integrado a esta branch.
- **develop:** Representa o ambiente de homologação (QA), contendo funcionalidades já integradas e prontas para validação.
- **feature/*:** Branches destinadas ao desenvolvimento de novas funcionalidades, derivadas da branch develop.

6.1.3.3 Política de Commits e Integração

- Cada nova funcionalidade é desenvolvida em uma branch do tipo feature/*, com commits frequentes e descritivos.
- Após conclusão, a feature é integrada à branch develop por meio de pull requests, passando por revisão de código.
- A branch develop é utilizada para testes no ambiente de homologação.
- Quando validado, o código é promovido para a branch main, sendo então disponibilizado em produção.

6.1.4 Análise dos Testes

A análise de teste será baseada na comparação entre os resultados esperados do teste e o resultado do teste em si. Com base nessa comparação, no caso de resultados não esperados ou insatisfatórios, haverá uma investigação de falhas, correção de erros, medição de desempenho, retestagem e então documentação dos resultados.

Ainda não foi obtido nenhum teste, pois o desenvolvimento do projeto está em suas fases iniciais.

6.2 Roteiro de teste:

Para minimizar os riscos no ambiente de teste e preservar a integridade do projeto, todos os testes planejados serão realizados em uma branch dedicada. Essa estratégia garante que possíveis erros ou modificações durante os testes não impactem o código principal do projeto.

Pré-condição para testes: fica determinado fazer na ordem dos códigos, assim, tudo estará pronto para o próximo passo.

Tabela 13 - Teste Unitário 1

Campo	Conteúdo
Código	TU01
Nome	Login com credenciais
Objetivo	Verificar se a função de autenticação retorna sucesso com credenciais válidas
Nível	Unitário
Tipo	Funcional
Precondições	Método de login implementado e usuário válido cadastrado
Estado	n/a
Resultados	Previsto: autenticação bem-sucedida / Realizado: n/a
Reparos	n/a
Ciclos	n/a

Fonte: Elaborado pelo autor (2026)

Tabela 14 - Teste Integrado 1

Campo	Conteúdo
Código	TI01
Nome	Integração entre backend e banco de dados
Objetivo	Verificar se o sistema armazena corretamente dados do banco

Nível	Integração
Tipo	Funcional
Precondições	Banco de dados ativo e conexão configurada
Estado	n/a
Resultados	Previsto: autenticação bem-sucedida / Realizado: n/a
Reparos	n/a
Ciclos	n/a

Fonte: Elaborado pelo autor (2026)

7. REFERÊNCIAS BIBLIOGRÁFICAS

ABRASEL. **Solução KDS: ferramenta inovadora para auxiliar restaurantes.** Disponível em:

<https://abrase.com.br/noticias/noticias/solucao-kds-ferramenta-inovadora-para-auxiliar-restaurantes/>.

ALELO. **Qual o melhor tipo de comanda para restaurante?** Disponível em:

<https://www.alelo.com.br/blog/estabelecimentos-comerciais/qual-o-melhor-tipo-de-comanda-para-restaurantes>.

CLOUDFY. **Os desafios na gestão de pedidos em bares e restaurantes.** Disponível em:

<https://www.cloudfy.net.br/blog/os-desafios-na-gestao-de-pedidos-em-bares-e-restaurantes.html>.

ECLETICA. **E-Garçom: Transformando o Atendimento em Restaurantes.** Disponível em:

<https://ecletica.com.br/e-garcom-transformando-o-atendimento-restaurantes/>.

NOX. **Sistema KDS: O que é, para que serve e como otimiza sua cozinha e cafeteria?**

Disponível em: https://nox.com.br/o-que-e-sistema-kds/?utm_medium=desktop.

OLITECNICA. **KDS: A revolução na gestão de pedidos para restaurantes e delivery.**

Disponível em:

<https://www.olitecnica.com.br/post/kds-a-revolucao-na-gestao-de-pedidos-para-restaurantes-e-delivery>.

THEFORKMANAGER. **Restaurant Table Turnover Rate Optimization.** Disponível em:

<https://www.theforkmanager.com/en/blog/restaurant-management/restaurant-table-turnover-tips-efficiency>